Code execution

Calepin runs the code in your document and renders the results in place. The simplest way to run code is a fenced block named after its language.

Code blocks

Write an ordinary Typst fenced block and name it after a language. *Calepin* runs the block and shows its output below the source:

````
```python
print(40 + 2)
```
````

```python
print(40 + 2)
```

```
42
```

Each language runs in a persistent session, so a variable set in one block is available in later blocks:

````
```python
x = 41
```

```python
print(x + 1)
```
````

```python
x = 41
```

```python
print(x + 1)
```

```
42
```

Inline code

An inline expression drops a computed value into a sentence. Use `#calepin.inline(...)`, usually through a short alias:

````
#let py = calepin.inline.with("python")

The answer is #py[`print(40 + 2)`].
````

The answer is 42.

Supported languages

Calepin has built-in engines for **Python** and **R**, and built-in diagram engines for **Mermaid**, **Graphviz DOT**, **TikZ**, and **D2**.

Any language with a Jupyter kernel also works: use the kernel name as the block language. Popular examples include **Bash** (`bash`), **Julia** (`julia`), **Octave** (`octave`), **Gnuplot** (`gnuplot`), and **Ruby** (`ruby`). Install kernels as described in the Jupyter install section.

Run `jupyter kernelspec list` to see what is registered. Whatever name appears there can be used as a block language directly:

```bash
echo "hello from bash"
```

Options

Options can be set in three places: as document-wide defaults, as arguments to one chunk, or as `#|` header lines inside a block.

Document defaults

`#calepin.setup` sets defaults for every chunk in the document:

```
#calepin.setup(echo: true, eval: true, results: "verbatim")
```

Chunk arguments

`#calepin.chunk(...)` overrides options for a single chunk. Pass the body as a fenced block and *Calepin* infers the engine from the fence:

```
#calepin.chunk(echo: false, results: "typst")[
```python
print("#strong[42 in Typst]")
```
]
```

42 in Typst

`#|` header lines

You can also place options at the top of a plain fenced block, one per line, each prefixed with `#|`:

```
```r
#| echo: false
#| fig-align: right
plot(mpg ~ hp, data = mtcars)
```
```

The `#|` form keeps options next to the code. Its downside is outside *Calepin*: compiling the `.typ` file directly with `typst` shows the `#|` lines as text inside the code block, while options passed to `#calepin.chunk(...)` do not.

Options reference

Global and chunk options

These options can be set in `#calepin.setup` as document-wide defaults and overridden per chunk.

Option	Default	Meaning
echo	`true`	Show the chunk's source code in the rendered document.
eval	`true`	Execute the code. When `false`, nothing runs and no output is produced (the source can still be shown via `echo`).

error	`false`	When `true`, capture an execution error and render it as output. When `false`, an error in the chunk aborts the build.
warning	`true`	Include warnings emitted by the engine in the output. When `false`, they are suppressed.
message	`true`	Include informational messages emitted by the engine (for example R's `message()` output). When `false`, they are suppressed.
results	`"render"`	How results are shown: `render` (pretty display of values, images, and tables), `verbatim` (raw output in a code block), `typst` (treat output text as Typst markup and render it), or `hide` (run the code but omit its output).
fig-device-format	`"svg"`	Format for figure files written by the engine: `svg`, `png`, `jpeg` (alias `jpg`), or `pdf`. Diagram engines always emit `svg` regardless of this setting.
fig-device-dpi	`150`	Resolution in dots per inch for raster formats (`png`, `jpeg`). Ignored for vector formats (`svg`, `pdf`).
fig-device-width	`6`	Width of the plotting device, in inches.
fig-device-height	`"auto"`	Height of the plotting device, in inches. `auto` derives it from the width and `fig-device-aspect`.
fig-device-aspect	`0.618`	Height-to-width ratio used when `fig-device-height` is `auto`: device height = `fig-device-width` × `fig-device-aspect`.
fig-width	`"70%"`	Width of the figure as rendered in the document. Accepts a Typst length or ratio (for example `70%` or `12cm`) or `auto`.
fig-height	`"auto"`	Height of the figure as rendered in the document. Accepts a Typst length or `auto`.
fig-align	`"center"`	Horizontal alignment of the figure in the document: `left`, `center`, or `right`.
fig-responsive	`true`	HTML output only: allow the figure to shrink to fit narrow viewports (sets `max-width: 100%`). No effect on paged output.

Chunk-only options

These options are understood only on individual chunks; they are not valid keys in `#calepin.setup`. The caption and layout options live here because they apply to one chunk's specific output.

Option	Default	Meaning
engine	inferred	Force the engine for this chunk instead of inferring it from the fence or surrounding context.
body	from fence	Provide the raw code body directly instead of writing a fenced block.
label	auto	Assign a stable chunk identifier used for cross-references and result lookup.
fig-caption	`none`	Caption text for the figure. When set, the output is wrapped in a numbered `figure` that can be cross-referenced.
fig-cap-location	`"auto"`	Where the caption sits relative to the figure: `top`, `bottom`, or `margin`. `auto` uses Typst's default placement.
fig-alt-text	`none`	Accessibility (alt) text for generated images. Empty when unset.
fig-subcaptions	`none`	Per-panel captions for a multi-image chunk, given as an array of strings (one per image, in order).
fig-layout-columns	`"auto"`	Column layout for a multi-image chunk: an integer number of equal columns, an array of explicit track sizes, or `auto` to choose a count from the number of images.
fig-layout-rows	`"auto"`	Row layout for a multi-image chunk: an integer number of equal rows, an array of explicit track sizes, or `auto`.

Quarto-style names

Chunk options have different names in *Calepin* and Quarto. Some Quarto aliases are accepted, but using them is not recommended, and *Calepin* emits a warning when it meets an unsupported name. The accepted aliases are:

- `out-width` maps to `fig-width`
- `out-height` maps to `fig-height`
- `out-align` maps to `fig-align`
- `fig-alt` maps to `fig-alt-text`
- `fig-subcap` maps to `fig-subcaptions`
- `fig-format` maps to `fig-device-format`
- `fig-dpi` maps to `fig-device-dpi`
- `layout-ncol` maps to `fig-layout-columns`
- `layout-nrow` maps to `fig-layout-rows`